Evolutionary Artificial Neuroidal Network Using Serverless Architecture

A Master's Thesis Presented to the Faculty of
the Department of Computer Science
Villanova University

In Partial Fulfillment of the
Requirements for the Degree of Master of Science
in
Software Engineering & Health Informatics

by
Chaitanya Kumar Mankala
August 2025

Under the Direction of
Dr. Ricardo Silva

Acknowledgment

This thesis would not have been possible without the support and guidance of my Professor, Dr. Ricardo Jose Silva, who has been a great mentor and leader throughout my thesis work and coursework under him.

Villanova University my “New Home” for giving me the opportunity to study and be part of our community, Dean Emory Woodard, Dr. Vijay Gehlot, Jessica Peoples, and the CLAS department for continued support and guidance.

To my fellow Villanovan students from CLAS whom I enjoyed working with during my Neuroid projects as part of coursework: Rachel Goeken, Lynn Yip, and Ali Abid. Thank you sincerely. It was a great learning experience, and I look forward to working with you in the future, as we may cross paths one day.

Contents

Abstract 5

Background 6

Methodology 28

Results 35

Parallel Processing and Data Visualization 35

Cloud Computing Overview 35

Key Service Models 35

Cloud Infrastructure 36

Core Components 36

Parallel Processing of EANN Using Serverless Architecture 37

Key Characteristics of Serverless Architecture 38

Key Components and Detailed Module Descriptions 38

Compute Infrastructure 38

Benefits of Serverless Architecture for EANN Processing 40

Workflow Summary 43

Security Layers 43

Data Visualization for EANN 44

Phase 1: On-Premise Implementation Using MVC Three-Tier Architecture 44

Overview 44

Architecture Components and Functionality 44

Phase 2: Cloud Migration - Neuroid in Cloud 46

Overview 46

Cloud Architecture Components and Functionality 47

End-to-End Workflow in Cloud Implementation 49

Benefits of Cloud Migration 50

Significance 57

Conclusion 58

Recommendations for Future Work 60

References 63

Appendix A 66

Appendix B 69

Appendix C 70

Appendix D 72

Appendix E 75

Appendix F 77

Glossary 83

Abstract

This thesis aims to address the limitations inherent in traditional Perceptron-based Artificial Neural Networks (ANNs), which are confined to linearly separable problems. An Evolutionary Artificial Neuroidal Network (EANN) is proposed to overcome these constraints, inspired by biological neural systems. Unlike the Perceptron, neuroids incorporate five adjustable parameters, offering greater flexibility and complexity in learning non-linear tasks. The research introduces a serverless cloud-based architecture to enable scalable and cost-efficient deployment of EANNs. Phase 1 implements an on-premise Model-View-Controller (MVC) three-tier architecture using Flask and Python for data processing and API management. Phase 2 transitions to the cloud, utilizing serverless AWS services, such as Lambda, Step Functions, and Batch to facilitate parallel processing and scalable model execution. A genetic algorithm is integrated to optimize iterative learning. Future work will further optimize cloud resource utilization, expand EANN's applicability to complex AI domains, and advance real-time, data-driven health informatics applications.

Background

The study primarily focuses on the development of an Evolutionary Artificial Neuroidal Network (EANN) and the implementation of serverless architecture to address the growing demand for scalable, efficient, and robust machine learning infrastructure. The proposed solution takes advantage of serverless technologies, enabling seamless parallel processing of multiple Artificial Neuroidal Networks (ANNs). This architecture enhances adaptability, reduces costs, and improves operational performance, significantly benefiting organizations tackling complex computational tasks.

A key innovation of the EANN lies in its use of neuroids instead of traditional perceptrons as the core computational units. Neuroids are designed to better emulate the behavior of biological neurons by incorporating five adjustable parameters, which provide added flexibility compared to the weight-based connections used in traditional ANNs. This flexibility allows EANNs to model complex, non-linear systems more effectively, overcoming the limitations of traditional networks that are primarily focused on linearly separable problems.

Another important feature of the EANN is its time-dependent output mechanism. Unlike conventional networks, which typically generate static outputs, EANNs produce outputs that vary over time, reflecting dynamic behaviors. This time variance enables the network to adapt its responses as input data evolves, making it particularly well-suited for applications requiring continuous learning, such as real-time data processing.

In terms of training, EANNs employ an evolutionary algorithm, as opposed to traditional gradient descent methods. Evolutionary algorithms simulate natural selection processes, optimizing the network's parameters through iterative steps such as mutation, crossover,

and selection. This approach fosters self-learning, allowing the network to refine its performance dynamically based on real-time data and environmental changes, improving its ability to solve complex, non-linear problems. Using state-of-the-art data visualization techniques by implementing innovative technologies such as React, Angular, SQL, PostgreSQL, and Python data processing and presentation are achieved.

Artificial Neural Networks (ANNs), in general, are modeled after biological neural systems. In ANNs, connections between neurons are represented as weighted links between nodes. Typically, ANNs consist of three main layers: the input layer, which captures raw data; hidden layers, which perform complex transformations to extract abstract features; and the output layer, which provides the final processed output, such as binary classifications or multi-class predictions. Deep neural networks (DNNs), which have multiple hidden layers, offer enhanced learning capabilities and can recognize more complex patterns, though they require substantial training datasets and computational resources.

Within the realm of ANNs, there are two common types: Feedforward Neural Networks (FNNs) and Convolutional Neural Networks (CNNs). FNNs are characterized by a simple, unidirectional flow of data from the input to the output layer and are typically used for tasks like classification and regression. On the other hand, CNNs, which are widely used in image and video recognition, utilize convolutional layers to automatically extract hierarchical features from the input data, improving the network's ability to detect patterns and spatial hierarchies.

The introduction of the EANN, with its neuroid-based computational units, time-dependent outputs, and evolutionary algorithm-based training, represents a significant advancement in artificial intelligence. By overcoming the limitations of traditional Perceptron-based

ANNs, EANNs offer greater flexibility and adaptability, making them better suited to handling complex, non-linear problems across a variety of applications.

Comparison Between Perceptron and Neuroid

The Perceptron: Foundational Yet Limited

The perceptron, introduced by Frank Rosenblatt in 1958, marks a seminal development in the field of artificial intelligence as one of the earliest artificial neural network (ANN) models. Operating as a binary classifier, the perceptron processes input features (x1,x2,……..,xn) through a weighted summation (w1x1+w2x2+…+b), where b represents the bias. The result is passed through a step activation function to produce a binary output, signifying its decision boundary. While its simplicity made it a compelling starting point for AI research, the perceptron is fundamentally constrained by several limitations:

1. Linear Decision Boundaries: The perceptrons assume data separability via a linear hyperplane, rendering them ineffective for modeling non-linear relationships.

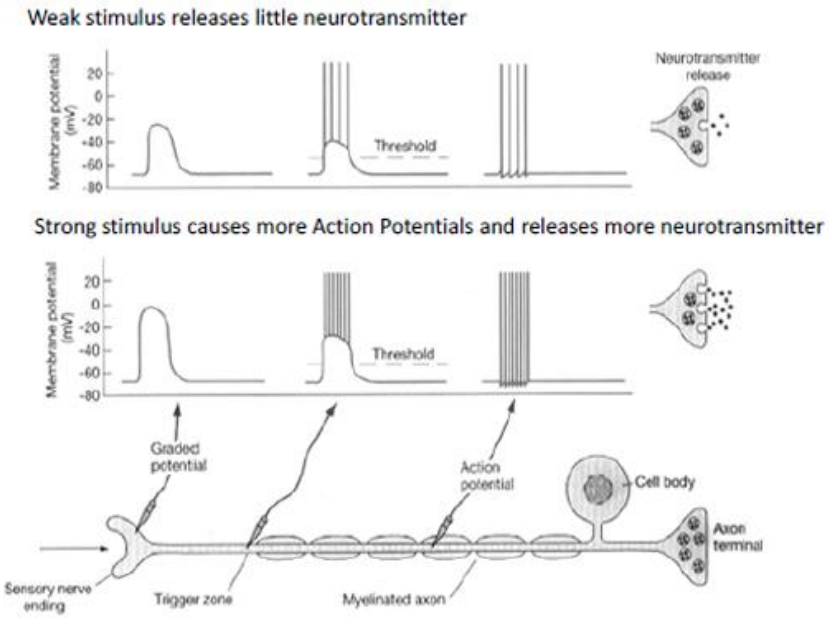
2. Single-Layer Architecture: The lack of depth in perceptron-based models prevents them from extracting hierarchical features or multiple levels of abstraction.

3. Gradient-Based Optimization: Perceptrons rely on static optimization techniques, which are inadequate for problems involving dynamic data.
4. Salability Issues: For large and complex data sets, the computational requirements for training perceptrons increase disproportionately, leading to inefficiencies. These limitations severely restrict the application of perceptrons to complex real-world problems, such as time-series prediction, image recognition, or health informatics, which often involve non-linear and high-dimensional data.

The Neuroid: An Evolutionary Advancement

The neuroid model (Argüello et al., 2012) introduces a more complex structure, mimicking the biological neuron's function. The neuroid performs three primary operations: comparison of inputs, pulse modulation, and pulse demodulation. The incoming set of inputs is processed through an execution block that performs cumulative computation and checks whether the sum signal exceeds the minimum threshold required for a valid output. If the signal is below the threshold, it is discarded. If the signal exceeds the threshold, further processing occurs, including pulse modulation and demodulation, before producing the final output. This structure allows the neuroid to process and transmit information more efficiently, offering potential improvements over traditional neural networks as shown in Figure 1.

Figure 1 The influence of stimulus intensity on firing frequency (Modified from (Silverthorn, D.U., 1998))



(Argüello et al., 2012, p. 1235)

In Dr. Argüello's and Rachel's thesis, the neuroid is described as performing three core operations to process incoming information. These operations are outlined as follows:

1. Comparison: The incoming inputs are compared to an action potential, a process like the function of the Perceptron model in artificial neural networks.

2. Pulse/Frequency Modulation: Upon comparison, if the input reaches the threshold of the action potential, the axonal signal (denoted as \(y(t)\) in Figure 3) outputs a value of 1; otherwise, it outputs a value of 0.

3. Pulse/Frequency Demodulation: The sequence of binary values (1's and 0's) produced by the axonal signal is converted from digital to analog, yielding a continuous output that is represented as a time versus frequency graph.

The neuroid, the foundational unit of Evolutionary Artificial Neuroidal Networks (EANNs), represents a paradigm shift from the simplicity of perceptrons to a more sophisticated, biologically inspired computational model. Neuroids are designed to emulate the dynamic behaviors of biological neurons by introducing five adjustable parameters, which enable them to capture non-linear dynamics, excitatory and inhibitory interactions, and multi-dimensional feature representations.

Key characteristics of neuroids include:

1. Biological Realism: Neuroids incorporate inhibitory and excitatory dynamics, enhancing their ability to simulate biological neural systems.

2. Non-Linearity: By design, neuroids excel at modeling non-linear relationships, overcoming the constraints of perceptrons.

3. Evolvability: Neuroids leverage evolutionary algorithms for optimization, incorporating processes like mutation, crossover, and selection to dynamically refine network parameters and architectures.

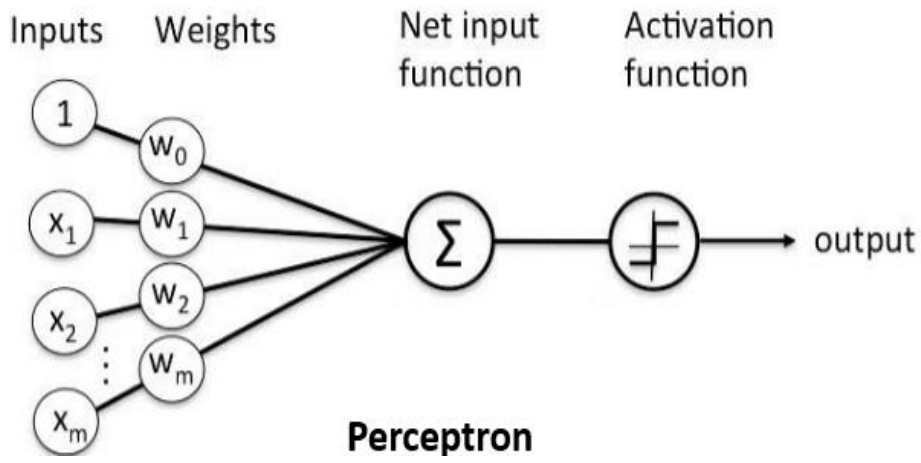
4. Adaptability: The neuroids' multi-parametric nature allows it to respond effectively to dynamic, real-time data, making it suitable for continuous learning environments.

In EANNs, neuroids are introduced as computational units that replicate the behavior of biological neurons more accurately than traditional artificial neurons in an FNN. The five adjustable parameters of neuroids are as follows:

- T = time interval (between two consecutive pulses measured in milliseconds (ms))
- Umbr = minimum threshold
- β = proportionality constant
- Kr = regeneration constant
- maxcount = maximum value for regeneration constant

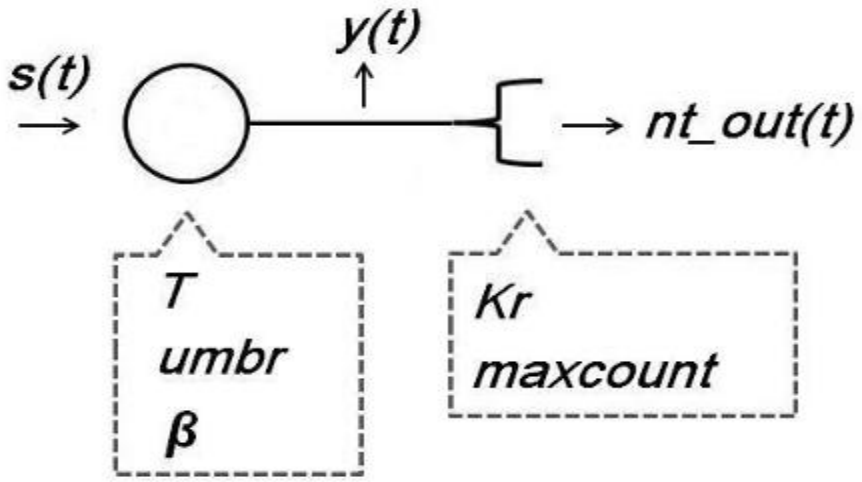
These parameters enable the EANN to model complex, non-linear systems more effectively, as they provide greater flexibility than traditional FNNs, allowing the network to better process real-world data that exhibits temporal and dynamic changes.

Figure 2 This diagram shows each operation as part of the perceptron:



The following diagram shows each operation as part of the neuroid:

Figure 3 The Neuroid, functional schema, and parameters



(Argüello et al., 2012, p. 1236)

Neuroid Model Diagram illustrate the signals s(t), y(t), and nt_out(t), where:

- s(t) represents the input signal as a function of time.
- y(t) is the output from the pulse frequency modulation (PFM) block.
- nt_out(t) is the output signal of the pulse frequency demodulation (PFDM).

In addition to these fundamental operations, neuroids are customizable through five key parameters: T, Umbr, β (Beta), Kr, and maxcount. Parameters Umbr and β are associated with the pulse frequency modulation (PFM) process, while Kr and maxcount are integral to the pulse frequency demodulation (PFDM) process. Each of these parameters allows for manipulation of the following:

- T: The time interval between two consecutive pulses, measured in milliseconds (ms).

- PFM Parameters:

  - Umbr: The minimum threshold.

  - β (beta): The proportionality constant.

- PFDM Parameters:

  - (Kr): The regeneration constant.

  - (maxcount): The maximum value for the regeneration constant.

Adjusting these parameters enables substantial computational efficiency with a minimal number of computational units. The key advantage of utilizing neuroids is that the weights of the incoming signals, represented by (nt_out(t)), are dynamically altered as a function of time, a feature not achievable with traditional neural operators.

Table 1: Comparison between Perceptron and Neuroid

Attribute	Perceptron	Neuroid
Mathematical Model	Weighted sum with step activation	Multi-parametric with five adjustable factors
Learning Approach	Static, gradient-based optimization	Dynamic, evolutionary algorithms
Problem-Solving Scope	Linear, binary classification	Non-linear, multi-faceted problem-solving
Scalability	Limited scalability with increased data size	Efficient parallelism and scalability
Adaptability	Fixed structure, static training	Adaptive, supports dynamic learning

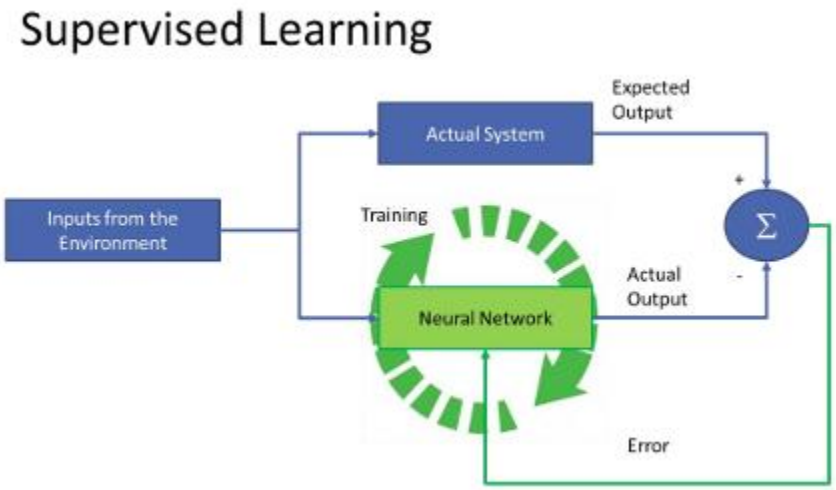
Artificial Neural Network Training

Training an artificial neural network involves teaching it to perform specific tasks by processing large datasets. During this process, the network learns patterns and relationships within the data, allowing it to generalize to new, unseen inputs. In supervised learning, labeled datasets guide the training process, providing correct output examples that the network uses to adjust its internal parameters. This iterative learning approach enables ANNs to improve their accuracy over time, making them highly effective for tasks such as classification, prediction, and adaptive control.

Neural network training is the process through which a network learns to perform specific tasks by processing large datasets. During the initial phase, the network analyzes both

labeled and unlabeled data, enabling it to generalize and accurately process unseen inputs. In supervised learning, labeled datasets, where the correct outputs are provided, are used to guide the network's learning process. These datasets serve as reference points for the network to adjust its internal parameters, thereby minimizing prediction errors and enhancing its ability to make accurate predictions on new, unseen data as shown in Figure 4

Figure 4 A representation of supervised learning for Artificial Neural Networks



The learning problem for neural networks revolves around identifying a set of parameters (weights) that minimize the loss function. Training algorithms are designed as a series of steps, with each step progressively reducing the loss by adjusting the network's parameters. The training process concludes when a predefined stopping criterion is met. Common training algorithms for traditional neural networks include:

- Gradient descent
- Newton method

- Conjugate gradient
- Quasi-Newton method
- Levenberg-Marquardt algorithm

Limitations of Traditional Perceptron Training Approaches for Neuroids

The traditional methods used to train perceptron-based networks, such as the perceptron learning rule and gradient-based optimization, are fundamentally inadequate for neuroid-based networks due to the following reasons:

1. Linear Assumptions: Perceptron training assumes linear relationships in data, which are insufficient for capturing the inherent non-linearities modeled by neuroids.

2. Local Optimization Challenges: Gradient-based approaches are prone to becoming trapped in local minima, a limitation that hinders the exploration of optimal solutions in complex, non-convex error surfaces. Neuroid-based networks, leveraging evolutionary algorithms, navigate such error landscapes more effectively.

3. Static Network Architectures: Traditional training methods are tailored for fixed architectures, lacking the flexibility to adapt to evolving data or dynamic tasks. Neuroids require adaptive learning mechanisms that can continuously evolve both structure and parameters.

4. Inefficiency in Large-Scale Applications: Perceptron-based networks face significant computational bottlenecks when applied to large datasets or complex networks. Neuroids, coupled with parallel processing, alleviate this constraint, enabling efficient training across distributed systems.

5. Inadequate for Multi-Parametric Optimization: Neuroids, with their five adjustable parameters, demand sophisticated optimization techniques that go beyond gradient descent. Evolutionary algorithms, designed to optimize complex systems, are better suited to the neuroids requirements.

While the perceptron has been instrumental in laying the foundation for artificial neural network research, its inherent limitations—linear decision boundaries, static learning methods, and lack of scalability—restrict its applicability to modern AI challenges. The neuroid, with its biologically inspired design and evolutionary learning mechanisms, addresses these challenges by introducing a flexible, adaptive, and non-linear framework for complex problem-solving. Traditional methods for training perceptrons are ill-suited for neuroids due to their inability to handle dynamic architectures, non-linear dynamics, and multi-parametric optimization. Neuroid-based networks, enabled by evolutionary algorithms and supported by advanced computational paradigms like parallel processing and serverless architectures, provide a robust and scalable solution for real-world, dynamic, and high-dimensional data problems.

Genetic Algorithms

Genetic Algorithms (GAs) are a subset of evolutionary algorithms inspired by the principles of natural selection and genetics, as introduced by Charles Darwin's theory of evolution. These algorithms mimic biological evolution to solve optimization and search problems, making them particularly effective for complex, multi-dimensional, and non-linear problem spaces. GAs operate on a population of potential solutions, iteratively evolving these solutions through processes analogous to natural genetic operations, including selection, crossover, and mutation which we will be using in EANN.

Core Principles of Genetic Algorithms

1. Population: A genetic algorithm begins with a randomly generated population of potential solutions, referred to as *individuals*. Each individual represents a candidate solution to the optimization problem.

2. Chromosomes: In GAs, each individual in the population is encoded as a *chromosome*. A chromosome is a data structure that encodes a candidate solution in a format suitable for genetic operations. This encoding can take various forms:

    - Binary Encoding: Chromosomes are represented as binary strings (e.g., 101), with each bit or group of bits representing a parameter of the solution.

    - Real-Valued Encoding: Chromosomes use real numbers to represent solution parameters, which is useful for continuous optimization problems.

    - Permutation Encoding: Chromosomes are ordered sequences, often used in combinatorial problems such as the traveling salesman problem.

- Tree-based encoding: Used in problems like symbolic regression or neural architecture search, where solutions are structured hierarchically.

3. Fitness Function: The fitness function evaluates how well each individual (chromosome) solves the problem at hand. This function assigns a fitness score to each candidate, determining its likelihood of being selected for reproduction. Higher fitness scores indicate better solutions.

4. Selection: Selection mimics the concept of survival of the fittest, where individuals with higher fitness are more likely to contribute their genes to the next generation. Common selection strategies include:

    - Roulette Wheel Selection: Individuals are selected probabilistically based on their fitness scores.

    - Tournament Selection: A subset of the population competes, and the fittest individual from this subset is chosen.

    - Rank Selection: Individuals are ranked by fitness, and selection is based on rank rather than absolute fitness values which is used in EANN in our research.

5. Crossover: During crossover, two-parent chromosomes combine to produce offspring. This operation promotes diversity in the population by mixing genetic material. Common crossover techniques include:

    - Single-Point Crossover: A single crossover point is chosen, and segments of the parents' chromosomes are exchanged.

- Two-Point Crossover: Two crossover points are chosen, and the segments between them are swapped.
- Uniform Crossover: Each gene in the offspring is independently chosen from one of the parents.

6. Mutation: Mutation introduces small random changes to chromosomes, helping to maintain genetic diversity and prevent premature convergence to suboptimal solutions. For example, flipping a bit in binary encoding or altering a parameter slightly in real-valued encoding.
7. Replacement: After generating offspring, the population is updated by replacing some or all individuals from the previous generation. Replacement strategies include generational replacement (entire population replaced) and elitism (retaining top individuals for the next generation).
8. Termination: The algorithm iterates through generations until a stopping criterion is met, such as reaching a maximum number of generations, achieving a desired fitness level, or observing stagnation in improvement.

Programmatically, Neuroid Networks are defined with three matrices, called chromosomes. Together, these are called the Network Genome. From a computational perspective, an Artificial Neuroidal Network is defined as a collection of three matrices, or chromosomes: Nodes, Connections, and Values. These three matrices encapsulate the total network parameters and form the Artificial Neuroidal Network Genome (ANNG). Specifically:

- The N-chromosome defines the number of neuroids in the network and their respective layers.

- The C-chromosome specifies the connectivity between neuroids.

- The V-chromosome defines the specific values of Umbr, β(beta), Kr, and maxcount for each neuroid in the network.

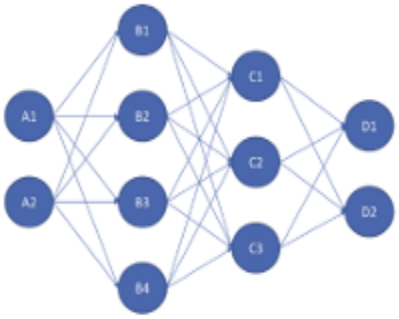
1. N-chromosome – defines how many neuroids there are and to what layer they belong.

2. C-Chromosome – defines the connections between neuroids.

3. V-Chromosome – defines the parameters (Umbr, β, etc.) for each neuroid.

As shown in the Figure 5

Figure 5 Neuroid network representation with its complete genome



V-Chromosome

Neuroid	Umbr	Beta	Kr	Maxcount
A1	0.43	2.00	3.00	97.00
A2	0.02	0.00	-5.00	36.00
B1	0.25	4.00	-3.00	70.00
B2	1.00	4.00	-5.00	39.00
B3	0.09	0.00	-4.00	11.00
B4	0.13	0.00	-2.00	69.00
C1	0.87	3.00	3.00	92.00
C2	0.68	5.00	-4.00	28.00
C3	0.09	4.00	-1.00	47.00
D1	0.55	3.00	-1.00	36.00
D2	0.48	3.00	-4.00	93.00

C-Chromosome

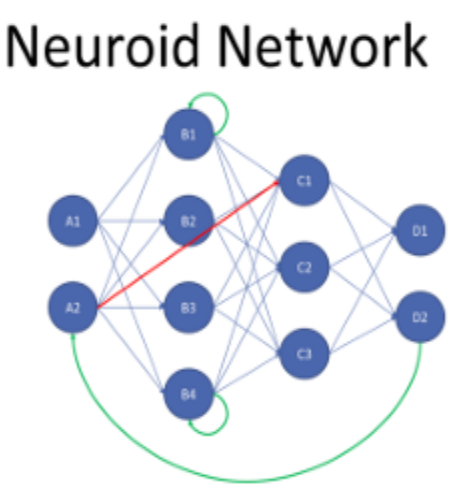
	A1	A2	B1	B2	B3	B4	C1	C2	C3	D1	D2
A1			1	1	1	1					
A2			1	1	1	1					
B1							1	1	1		
B2							1	1	1		
B3							1	1	1		
B4							1	1	1		
C1										1	1
C2										1	1
C3										1	1
D1											
D2											

N-Chromosome

N	A	B	C	D
1	1	1	1	1
2	1	1	1	1
3		1	1	
4		1		

By modifying the C-Chromosomes, connections can be added or deleted from the network as shown in Figure 6. From the Figure 6, three feedback loops (green), and one forward propagating connection (red) have been added to the network by modifying one single chromosome. Feedback loops are very important to generate memory or pacing circuits.

Figure 6 Neuroid network representation feedback and forward propagation connections



N-Chromosome

N	A	B	C	D
1	1	1	1	1
2	1	1	1	1
3		1	1	
4		1		

V-Chromosome

Neuroid	Umbr	Beta	Kr	Maxcount
A1	0.43	2.00	3.00	97.00
A2	0.02	0.00	-5.00	36.00
B1	0.25	4.00	-3.00	70.00
B2	1.00	4.00	-5.00	39.00
B3	0.09	0.00	-4.00	11.00
B4	0.13	0.00	-2.00	69.00
C1	0.87	3.00	3.00	92.00
C2	0.68	5.00	-4.00	28.00
C3	0.09	4.00	-1.00	47.00
D1	0.55	3.00	-1.00	36.00
D2	0.48	3.00	-4.00	93.00

C-Chromosome

	A1	A2	B1	B2	B3	B4	C1	C2	C3	D1	D2
A1			1	1	1	1					
A2			1	1	1	1	1				
B1			1				1	1	1		
B2							1	1	1		
B3							1	1	1		
B4						1	1	1	1		
C1										1	1
C2										1	1
C3										1	1
D1											
D2		1									

The ANNG is a new concept that we want to introduce as part of the Neuroid Network System and it becomes a building block to use genetic algorithms, as a form of training, which currently is not a standard approach. A genetic algorithm is a metaheuristic inspired by the process of natural selection that belongs to the larger class of evolutionary algorithms. Genetic algorithms are commonly used to generate high-quality solutions for optimization and search problems by relying on biologically inspired operators such as mutation, crossover, and selection (Mitchell, 1996). Genetic algorithms rely on operations such as mutation, crossover, and selection, to evolve from one generation to the other. To the best of our knowledge genetic algorithms have not been utilized to train Artificial Neural Networks, but in our case they are particularly well suited for this purpose.

Advantages of Genetic Algorithms for Neuroid-Based Systems

1. Adaptability to Complex Landscapes: Unlike gradient-based optimization, which often gets trapped in local minima, GAs explore the solution space broadly, making them well-suited for non-linear and multi-modal error surfaces characteristic of neuroid-based networks.

2. Dynamic Evolution: GAs allow continuous evolution of neuroid parameters, adapting the network to changing data patterns and ensuring long-term efficiency.

3. Scalability: The parallel nature of GAs aligns seamlessly with serverless architectures, enabling efficient optimization of large-scale networks.

4. Robustness: GAs are less sensitive to initial parameter settings, ensuring better convergence for complex, high-dimensional optimization tasks.

Genetic algorithms provide a robust framework for optimizing complex systems like neuroid-based EANNs by leveraging the principles of evolution. Chromosomal representation serves as a versatile encoding mechanism, capturing the multi-parametric nature of neuroids and enabling dynamic adaptation through evolutionary operators. By overcoming the limitations of traditional training methods, GAs enhance the scalability, efficiency, and adaptability of neuroid-based networks, paving the way for solving intricate, real-world problems with unparalleled precision.

Matrix Operations

Feedforward Neural Networks and Evolutionary Artificial Neuroid Networks

Matrix operations, particularly matrix multiplication and transposition, form the backbone of the computational processes in both Feedforward Neural Networks (FNNs) and Evolutionary Artificial Neuroid Networks (EANNs). These operations are essential for data transformation through the layers of the network, facilitating the flow of information and enabling the model to learn complex patterns from input data. The proper execution of these operations ensures that the network processes inputs efficiently, adjusts its internal parameters during training, and generates accurate outputs, which is central to the learning dynamics of neural network models.

Problem Statement

The traditional Perceptron-based Artificial Neural Networks (ANNs) have been a foundational approach in artificial intelligence; however, they exhibit significant limitations in addressing complex, non-linear real-world problems. Perceptrons are inherently constrained by their linearity assumption, making them suitable only for simple, binary classification tasks where data is linearly separable. This design limitation hinders their effectiveness in solving more intricate, multi-dimensional problems, such as time-series forecasting, image recognition, and health informatics. The inability to model complex, non-linear relationships in data significantly restricts their applicability in dynamic, high-dimensional environments. Furthermore, Perceptron struggles with scalability and adaptability when handling large datasets or evolving data streams, often resulting in prolonged training times and a lack of generalization over time.

To overcome these limitations, the proposed solution involves the use of Evolutionary Artificial Neuroidal Networks (EANNs), which introduce a more flexible, biologically inspired approach to neural computation. EANNs utilize neuroids—computational units modeled after biological neurons—that are governed by five adjustable parameters, providing greater flexibility and efficiency in modeling complex, non-linear relationships in data. In contrast to Perceptrons, neuroids facilitate more nuanced interactions between layers, enabling the model to learn and adapt continuously to dynamic data. This evolutionary approach to training, which leverages evolutionary algorithms such as mutation, crossover, and selection, enhances the network's ability to handle complex optimization tasks and avoid the pitfalls of local minima, making it more suited for real-time learning and generalization.

However, despite the enhanced capabilities of EANNs, the implementation and scalability of these networks in real-world applications face significant challenges due to the high computational demands of training and inference. Traditional server-based infrastructures are ill-equipped to handle the large-scale, dynamic processing requirements of EANNs, resulting in inefficiencies and resource constraints. To address these issues, the use of serverless architecture combined with parallel processing offers a transformative solution. Serverless computing provides an elastic, on-demand computing environment that can dynamically allocate resources based on demand, ensuring efficient scaling without the need for extensive infrastructure management. Parallel processing further accelerates the training process by distributing computational tasks across multiple nodes, enabling EANNs to process large datasets more efficiently, reducing latency, and increasing throughput.

Additionally, the integration of real-time data visualization techniques within the cloud-based infrastructure enhances the transparency and interpretability of EANNs. Traditional neural networks, including Perceptrons, often cannot provide meaningful insights into their internal decision-making processes, making it difficult for users to trust and understand the model's predictions. By incorporating data visualization, the proposed EANN model allows stakeholders to monitor the learning process, fine-tune parameters, and gain actionable insights, improving decision-making and model performance.

In conclusion, the limitations of traditional Perceptron-based ANNs, such as their inability to model non-linear relationships and scale efficiently, are addressed through the development of Evolutionary Artificial Neuroidal Networks (EANNs). The combination of EANNs with serverless architecture, parallel processing, and data visualization offers a scalable, cost-effective, and transparent solution to complex, real-time data analysis tasks. This approach provides a more flexible and robust framework for solving modern AI

problems, particularly those requiring continuous learning, adaptation, and real-time processing.

Drawing on the foundational work of Argüello et al. (2012), and Rachel et al. (2022) this thesis seeks to explore the following research questions:

- Can neuroids and basic neuroid networks be replicated using Python?

- Is it possible to generate variable and scalable neuroid network architectures using Python and represent them in a user interface using JavaScript, Flask, Python, PostgreSQL, and React on-premise and cloud?

- Can genetic algorithms be effectively employed to train a neuroid network?

- Can a feedforward approach address efficient learning, handling non-linear relationships, and parallel processing of inputs?

- Can cloud-based serverless architecture solve the parallel processing of EANN, and scalability to analyze and train the network?

- Can a data visualization framework be used for processing and presenting results?

Methodology

Methodology for Implementing Genetic Algorithm, Parallel Processing of Evolutionary Artificial Neuroidal Networks (EANN) on AWS Cloud Computing Using Serverless Architecture and Data Visualization

This methodology outlines the step-by-step approach for the design and implementation of Genetic Algorithm (GA), Parallel Processing of Evolutionary Artificial Neuroidal Networks (EANN), and Data Visualization across two distinct phases: Phase 1: On-Premise Architecture and Phase 2: Cloud Migration to AWS - Neuroid in Cloud. The primary objective of this work is to leverage AWS cloud computing capabilities for scalable, efficient parallel processing of EANN using a serverless architecture, while also providing a comprehensive and interactive user interface for data visualization and analysis.

Phase 1: On-Premise Architecture

Initial Implementation with MVC Three-Tier Architecture

1. Genetic Algorithm Implementation

In the initial phase, the Genetic Algorithm (GA) is implemented to optimize the performance of the Evolutionary Artificial Neuroidal Network (EANN). The implementation follows a series of steps designed to mimic the principles of natural evolution, including:

- Initialization: The GA begins by generating an initial population of potential solutions, represented by the weights and biases of an ANN model. These solutions will evolve over several iterations.

- Fitness Evaluation: A fitness function is defined to evaluate the performance of each solution based on its ability to correctly solve the problem, typically using training and validation datasets. The fitness score will guide the selection of the most suitable solutions.

- Selection, Crossover, and Mutation: The process of natural selection is simulated by selecting individuals with higher fitness scores, which are then crossed over and mutated to create the next generation of potential solutions. This process is repeated iteratively until the stopping criteria are met.

The GA will be used to refine the performance of the ANN through these evolutionary steps, aiming for improved results in tasks such as classification, regression, or optimization.

2. Parallel Processing of EANN Using MVC Three-Tier Architecture

In this phase, the system follows a Model-View-Controller (MVC) three-tier architecture to separate the concerns of data, business logic, and presentation. The architecture is implemented as follows:

- Presentation Layer (Front-End): The user interface is designed using HTML, JavaScript, and Flask. This front-end layer provides an intuitive and interactive experience for users to upload data, monitor model training, and view the results. The UI allows users to initiate data uploads and track the evolution of the GA and ANN.

- Business Logic and API Layer (Middle Layer): The middle layer is built with Flask, which serves as the backend API. Flask handles requests from the front end, triggers the GA training processes, and facilitates communication with the data

layer for model processing. This layer encapsulates the core logic for handling model training, evaluation, and optimization through GA.

- Data Layer: The data layer is implemented using Python for processing, manipulation, and storage of training data. Excel is used as an intermediary format for data storage, where datasets are cleaned, normalized, and structured before being passed on to the GA for training and evaluation.

3. Data Visualization

Data visualization in Phase 1 is crucial for tracking the optimization and performance of the models. The following steps outline the process:

- Data Preparation: The results from each GA iteration, including fitness scores and model performance, are gathered, and stored for visualization.

- Visualization Tools: JavaScript libraries such as D3.js or Plotly.js are employed to create dynamic, interactive visualizations. These visualizations include charts, graphs, and heatmaps that represent the evolution of the GA process and the performance metrics of the ANN.

- Real-Time Feedback: As the training progresses, visual updates are provided to the user, showing the optimization journey, convergence trends, and model performance.

This visualization component is essential for enabling stakeholders to assess the effectiveness of the evolutionary process and make informed decisions about the training and deployment of the ANN.

Phase 2: Neuroid in Cloud – Cloud Migration

The second phase involves migrating the existing system to the AWS cloud to leverage the scalability, elasticity, and flexibility of cloud-based resources. This phase introduces a cloud-native architecture that utilizes AWS serverless technologies to optimize the execution of the Genetic Algorithm and parallel processing of EANN models.

1. Cloud-Based Genetic Algorithm Implementation Using AWS Services

In this phase, the GA will be adapted to run on AWS, leveraging a serverless architecture to dynamically allocate resources based on the needs of the application. Key AWS services will be utilized as follows:

- AWS Lambda: Serverless compute service that will run the GA's functions. Each step of the GA process (selection, crossover, mutation, and fitness evaluation) will be executed as an independent Lambda function, triggered automatically based on specific events such as data uploads or training requests.

- Amazon S3: AWS's highly scalable object storage service will be used to store large datasets (e.g., training data and model parameters) and processed results. S3’s versioning capabilities ensure that all versions of models are stored for tracking and reproducibility.

- Amazon SQS: Simple Queue Service (SQS) will manage asynchronous tasks for processing GA iterations. Once a task is completed (e.g., a model training step), the next task is queued, ensuring smooth orchestration of the entire process.

- AWS Step Functions: This service will orchestrate and coordinate multiple Lambda functions, creating workflows that define the order of operations in the GA process.

Step Functions will handle the sequencing of operations and ensure the execution flow is maintained for complex tasks involving multiple parallel operations.

- AWS EC2 Spot Instances: For large-scale GA tasks that require additional compute power, EC2 Spot Instances can be used to provision cost-effective, temporary compute resources. These instances will run parallel computations such as model training across multiple ANNs.

2. Parallel Processing of EANN Using AWS Serverless Architecture

- AWS Lambda: Lambda functions will be employed to handle the training and inference processes of the EANN. Each model training task will be initiated as a separate Lambda function, enabling parallel processing of multiple models simultaneously.

- AWS Batch: AWS Batch will be used for executing large-scale parallel processing tasks. It will manage the execution of batch jobs for model training, utilizing compute clusters to scale the workloads dynamically based on the system's requirements.

- Amazon SQS: SQS will ensure that training tasks are processed efficiently by managing the flow of requests and processing in a queue-based manner.

- Amazon RDS (PostgreSQL): The cloud-based database will manage large datasets for model training, storing structured data such as model parameters and results. PostgreSQL will be used as a relational database, ensuring high availability, scalability, and fault tolerance in cloud environments.

- AWS CloudWatch: For monitoring and logging, AWS CloudWatch will be used to track the performance of Lambda functions, SQS queues, and EC2 Spot Instances. CloudWatch provides detailed logs and metrics, facilitating performance optimization and troubleshooting.

3. Data Visualization in AWS Cloud

- Amazon S3: Data results such as GA fitness scores and ANN performance metrics will be stored in S3, making them accessible for visualization. S3 provides seamless integration with other AWS services to automate the visualization process.

- AWS Amplify and React: AWS Amplify will be used to build and deploy the front-end React application that interacts with AWS services for real-time data fetching and visualization. React will provide a responsive, dynamic user interface for interacting with the data and monitoring the evolution of the GA process.

- Amazon QuickSight: For advanced data visualization and analytics, QuickSight will be integrated into the system. It will allow users to generate business intelligence dashboards, providing insights into the optimization process, model accuracy, and GA progress.

- D3.js / Plotly.js: These JavaScript libraries will be used to create interactive, real-time visualizations of the GA process and EANN model performance. These visualizations will include charts, graphs, and heatmaps that dynamically update as new data is generated.

The methodology for implementing the Genetic Algorithm (GA), Parallel Processing of EANN, and Data Visualization is systematically divided into two phases:

- Phase 1 involves the development of an on-premise system using MVC three-tier architecture with Flask, Python, and Excel for local data processing, followed by the visualization of results using JavaScript libraries.

- Phase 2 introduces a cloud-native architecture, migrating the system to AWS and employing serverless technologies to scale the GA and parallel processing of EANN. The cloud infrastructure includes AWS Lambda for serverless execution, Amazon S3 for data storage, Amazon RDS (PostgreSQL) for database management, AWS Step Functions for orchestration, and AWS Batch for parallel processing. The presentation layer is built using React, while D3.js and Plotly.js provide data visualization.

This methodology ensures a comprehensive and scalable solution for implementing EANN and GA in a cloud environment using AWS, offering a flexible, cost-efficient approach to parallel processing while maintaining high performance and providing an interactive, user-friendly interface for data visualization and analysis. The migration to AWS enhances the system's capability to scale, manage large datasets, and execute computationally intensive tasks efficiently.

Results

Parallel Processing and Data Visualization

Cloud Computing for Parallel Processing of Evolutionary Artificial Neuroidal Networks (EANN) Using Serverless Architecture

Cloud Computing Overview

Cloud computing represents a transformative paradigm in modern information technology, offering on-demand access to computing resources such as servers, storage, databases, networking, and software over the internet. By abstracting the underlying hardware and infrastructure, cloud computing provides organizations with scalable, flexible, and cost-efficient solutions. It enables businesses to dynamically allocate resources, optimize performance, and reduce operational complexity, fostering innovation and agility.

Key Service Models

- Infrastructure-as-a-Service (IaaS): Provides virtualized computing resources over the internet, allowing users to deploy and manage their applications, operating systems, and software while the cloud provider handles the physical infrastructure.
- Platform-as-a-Service (PaaS): Offers a managed environment for application development, including tools and frameworks that streamline deployment and scaling. PaaS solutions abstract away infrastructure concerns, allowing developers to focus on coding and innovation.

- Software-as-a-Service (SaaS): Delivers fully functional applications accessible through web browsers. SaaS solutions eliminate the need for installation, maintenance, and updates, providing users with scalable, on-demand software solutions.

These service models collectively support diverse workloads, ranging from simple web applications to complex machine learning systems, including Evolutionary Artificial Neuroidal Networks (EANNs).

Cloud Infrastructure

Cloud infrastructure encompasses the core building blocks required to deliver cloud services, including computing power, storage, and networking components. For parallel processing of EANNs, cloud infrastructure must support high-performance computation, secure data storage, and reliable networking to ensure the seamless execution of complex workflows.

Core Components

- Compute Resources: Virtual machines, serverless functions, and containerized services provide the necessary processing power to handle computationally intensive EANN tasks. These resources can be dynamically scaled to accommodate varying workloads, ensuring optimal performance and cost-efficiency.

- Data Storage: Scalable object storage systems offer secure, durable repositories for raw datasets, preprocessed data, model checkpoints, and inference results.

These storage solutions support high availability, version control, and access management, ensuring data integrity and reliability.

- Networking: Virtual Private Clouds (VPCs), load balancers, and secure connectivity solutions facilitate reliable communication between compute resources and data storage. Advanced networking features, such as traffic encryption and firewalls, enhance security and performance.

- Security Framework: Identity and Access Management (IAM), encryption, and security groups provide granular access control and data protection. These security measures ensure compliance with regulatory standards and safeguard sensitive information.

Together, these components form a robust infrastructure that supports the parallel processing and deployment of EANN models.

Parallel Processing of EANN Using Serverless Architecture

Evolutionary Artificial Neuroidal Networks (EANNs) leverage evolutionary algorithms to optimize artificial neural networks (ANNs) by simulating natural selection processes. This involves iterative training, evaluation, and selection of neural network models across multiple generations. Given the computational intensity of this process, parallel processing is essential to accelerate training and improve efficiency.

Serverless architecture offers an ideal solution for parallel processing of EANNs by abstracting infrastructure management and dynamically scaling resources. Serverless computing allows developers to focus on writing code while the cloud provider manages the underlying infrastructure, ensuring seamless scalability and resource optimization.

Key Characteristics of Serverless Architecture

- Event-Driven Execution: Serverless functions are triggered by specific events, such as data uploads or task completions. This enables automated, responsive processing and efficient resource utilization.

- Stateless Functions: Each serverless function operates independently, allowing for horizontal scaling and parallel execution. Stateless design simplifies deployment and enhances fault tolerance.

- Automatic Scaling: Resources are provisioned and scaled automatically based on workload demands. This ensures that computational resources are available when needed, optimizing performance and cost-efficiency.

- Pay-as-You-Go Pricing: Serverless platforms charge based on actual usage, minimizing costs by eliminating the need for pre-allocated resources. This model ensures that organizations only pay for the compute time consumed during execution.

Key Components and Detailed Module Descriptions

Compute Infrastructure

- Serverless Functions (Function-as-a-Service): These lightweight, event-driven functions handle specific tasks within the EANN workflow, such as data preprocessing, evolutionary operations (mutation, crossover, selection), and model evaluation. Serverless functions are automatically scaled and executed in

parallel, ensuring efficient processing of large datasets and evolutionary populations.

- Batch Processing Services: Managed batch processing services distribute large-scale EANN training tasks across multiple compute nodes. These services break down complex workloads into smaller tasks, executed concurrently to accelerate processing. By leveraging spot instances, batch processing minimizes costs while maintaining high performance.

- Container Services: Containers provide consistent environments for EANN models across development, testing, and production stages. They support microservices architectures and enable efficient deployment and scaling of EANN components.

Data Storage and Management

- Scalable Object Storage: Centralized storage systems handle raw datasets, preprocessed data, and model artifacts. These storage solutions offer high durability, redundancy, and access control, ensuring data availability and integrity. Versioning capabilities support model reproducibility and traceability.

- Event-Driven Data Preprocessing: Data uploads to storage trigger serverless functions that automate preprocessing tasks, such as data cleaning, transformation, and augmentation. This ensures that data is consistently prepared for training and evaluation.

Workflow Orchestration and Management

- Stateful Orchestration Services: Cloud-native orchestration tools define and manage complex EANN workflows, including parallel execution of training tasks,

model evaluation, and evolutionary operations. Orchestration services ensure task synchronization, error handling, and conditional logic, enhancing workflow reliability.

- Message Queuing Services: These services facilitate task distribution and load balancing by managing the execution of concurrent tasks. Queuing systems ensure that resources are efficiently utilized and prevent bottlenecks in the workflow.

Networking and Security

- Virtual Private Cloud (VPC): Provides isolated environments for EANN processing, enhancing security and performance. VPCs enable secure communication between computing resources, storage systems, and external services.

- Identity and Access Management (IAM): Granular access control mechanisms ensure that only authorized users and processes can access sensitive data and resources. IAM policies and roles provide fine-grained permissions, enhancing security and compliance.

- Encryption and Firewall Controls: Data is encrypted both at rest and in transit, protecting it from unauthorized access. Firewall rules and security groups control traffic flow, preventing security breaches and ensuring network integrity.

Benefits of Serverless Architecture for EANN Processing

Scalability: Serverless platforms dynamically adjust compute resources based on workload demands, enabling seamless scalability. This ensures that parallel EANN

processing can handle fluctuating workloads without manual intervention, optimizing resource allocation and performance.

Cost Efficiency: Pay-as-you-go pricing minimizes infrastructure costs by charging only for the compute time consumed during execution. Leveraging spot instances for batch processing further reduces expenses compared to maintaining idle resources. This cost model ensures that organizations achieve maximum efficiency and return on investment.

Simplified Operations: Serverless architecture abstracts infrastructure management, allowing developers and data scientists to focus on EANN development and optimization. This reduces operational complexity and accelerates deployment cycles, enhancing productivity and innovation.

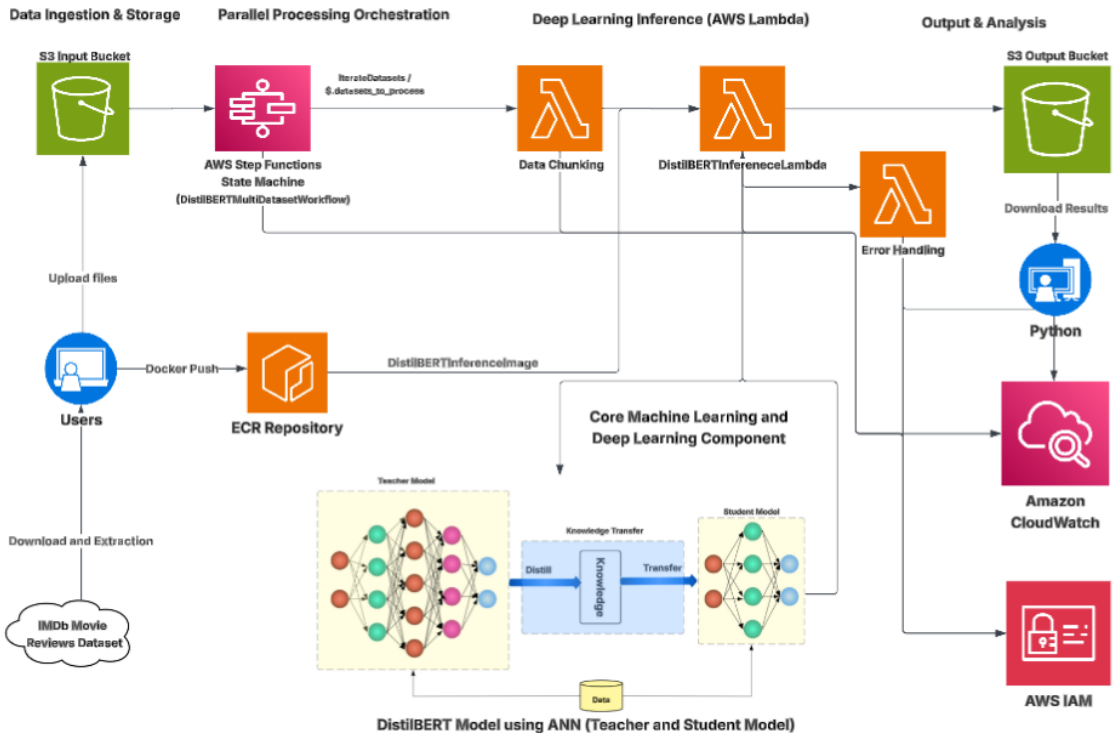
Enhanced Performance: Parallel processing of EANN tasks reduces training time and improves convergence rates. Managed orchestration services ensure that workflows run efficiently, with built-in error handling and retries for fault tolerance. This accelerates model development and enhances performance.

Security and Compliance: Cloud platforms provide robust security features, including IAM, encryption, and network isolation. These measures ensure data protection, compliance with regulatory standards, and secure access to resources. Serverless functions operate within isolated environments, minimizing security risks.

Cloud infrastructure combined with serverless architecture, offers a powerful, scalable, and cost-effective solution for parallel processing of Evolutionary Artificial Neuroidal Networks (EANNs). By leveraging managed compute, storage, and orchestration services, organizations can streamline their EANN workflows, accelerate model training, and drive innovation. This architecture not only enhances operational efficiency but also

provides robust security and compliance, making it an ideal choice for handling complex, computationally intensive machine learning tasks. The diagram uses DistilBERT teacher-student model using Deep Learning and parallel processing for sentiment analysis which is similar architecture framework for Artificial Neuroidal Network. As shown in Figure 7

Figure 7 Serverless architecture for evolutionary artificial neuroidal network for parallel processing



Workflow Summary

I. Data ingestion into S3 triggers Lambda functions for preprocessing.

II. Preprocessed data persisted in S3, and metadata was sent to SQS.

III. SQS messages activate a Step Functions workflow.

IV. The Step Functions workflow coordinates parallel training jobs on AWS Batch, utilizing cost-effective EC2 Spot Instances.

V. Trained models are saved to S3.

VI. Inference requests (e.g., via API Gateway) invoke Lambda functions, which load the model from S3 and execute inference.

VII. Inference results are stored in S3, and SNS notifications are triggered.

Security Layers

- VPC
- Security Groups
- WAP protecting API Gateway and Lambda
- Access management

Data Visualization for EANN

End-to-End Implementation of Parallel Processing of EANN with Serverless Architecture and MVC Three-Tier Structure

Phase 1: On-Premise Implementation Using MVC Three-Tier Architecture

Overview

The initial on-premise implementation follows a Model-View-Controller (MVC) architecture within a three-tier structure. This setup ensures separation of concerns, modularity, and maintainability, facilitating smooth development and future migration to the cloud. The system leverages HTML, JavaScript, and Flask for the presentation layer, Python and Excel for data storage and processing, and Flask for the business logic and API layer.

Architecture Components and Functionality

Presentation Layer (Frontend)

- Technologies: HTML, JavaScript, Flask
- Description: The presentation layer acts as the user interface, providing dynamic and interactive dashboards for visualizing data insights.
- Key Functionalities:

- Data Visualization: HTML and JavaScript render charts and graphs using libraries such as Chart.js or D3.js to represent the EANN processing results.

- Interactive UI Elements: Form handling, input validation, and dynamic content updates ensure a seamless user experience.

- Flask Integration: Facilitates the rendering of templates and serves as a bridge between the frontend and backend logic.

Middle Layer (Business Logic and API)

- Technologies: Flask, Python

- Description: This layer manages application logic, coordinating data processing and facilitating communication between the presentation and data layers.

- Key Functionalities:

  - API Development: RESTful APIs handle data requests and responses, ensuring efficient communication with the front end.

  - Data Processing Logic: Python scripts manage data transformation, validation, and preprocessing tasks.

  - Error Handling and Security: Robust error-handling mechanisms and authentication protocols ensure application reliability and data integrity.

Data Layer

- Technologies: Python, Excel
- Description: The data layer stores and manages raw and processed datasets, facilitating efficient data access and manipulation.
- Key Functionalities:
    - Data Storage: Excel files serve as the primary storage medium, holding training datasets, intermediate results, and final outputs.
    - Data Manipulation: Python libraries such as Pandas and NumPy perform data analysis and transformation, preparing data for EANN processing.
    - Data Integrity: Ensures consistency and accuracy during data processing and retrieval.

Phase 2: Cloud Migration - Neuroid in Cloud

Overview

The cloud migration phase, branded as Neuroid in Cloud, transforms the on-premise system into a robust, scalable, and cloud-native architecture. This phase adopts cloud services and modern web technologies, ensuring high performance, reliability, and scalability. The system architecture incorporates PostgreSQL for data management, React, and Flask for the presentation layer, and Python with Flask for the API and business logic layer.

Cloud Architecture Components and Functionality

Cloud Infrastructure Overview

- The cloud platform offers managed services for computing, storage, and data processing, ensuring dynamic scalability and high availability.
- Key infrastructure elements include virtual servers, containerized applications, and serverless functions to support parallel EANN processing.

Presentation Layer (Frontend in Cloud)

- Technologies: React, Flask
- Description: The cloud-based frontend leverages React to build dynamic and responsive user interfaces, enhancing the user experience.
- Key Functionalities:
  - Component-Based Design: React's modular components ensure reusable, maintainable code, facilitating faster development and updates.
  - API Integration: React communicates with Flask APIs to fetch data from the cloud database, ensuring real-time data updates.
  - Dynamic Dashboards: Advanced visualization tools display processing results, allowing users to interact with complex data insights effectively.

Middle Layer (Business Logic and API in Cloud)

- Technologies: Python, Flask
- Description: This layer remains the core processing engine, orchestrating data handling, processing, and API services.
- Key Functionalities:
    - Scalable APIs: RESTful endpoints facilitate seamless communication between the front end and the data layer, ensuring low latency responses.
    - Serverless Functions: Deployed in the cloud, these functions handle intensive data processing tasks, enabling parallel processing for enhanced performance.
    - Orchestration Logic: Manages workflow execution, error handling, and task coordination for parallel EANN training and inference.

Data Layer (Cloud Database)

- Technologies: PostgreSQL
- Description: PostgreSQL serves as a robust, relational database, providing scalable and reliable storage for large datasets, processed results, and model metadata.
- Key Functionalities:
    - Data Management: Supports complex queries and ensures data consistency through ACID compliance.

- Backup and Recovery: Automated backup and restore processes ensure data resilience.
- High Availability: Cloud infrastructure guarantees data availability and fault tolerance.

End-to-End Workflow in Cloud Implementation

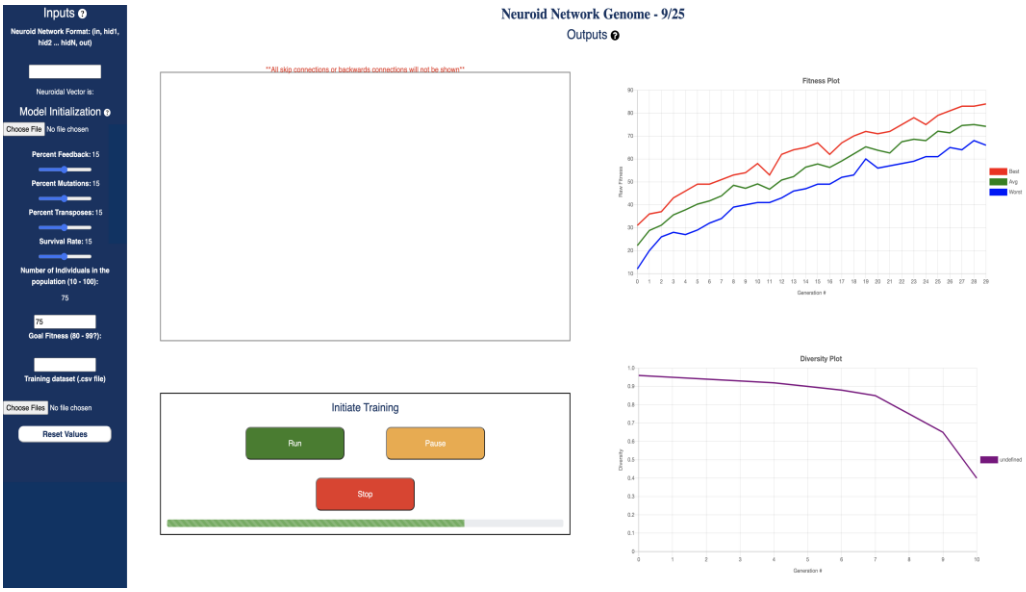
1. Data Ingestion and Preprocessing: Raw data is uploaded and preprocessed using serverless functions. Processed data is stored in PostgreSQL, ready for EANN processing.
2. Model Training and Parallel Processing: Serverless architecture enables parallel execution of EANN training jobs. The middle layer orchestrates these processes, ensuring efficient task distribution and error handling.
3. Data Access and Visualization: The front end retrieves processed results via Flask APIs and dynamically visualizes them using React. Users interact with dashboards to gain insights into model performance and predictions.
4. Continuous Monitoring and Notifications: Built-in cloud monitoring tools track system performance and trigger alerts, ensuring reliability and performance optimization.

Benefits of Cloud Migration

- Scalability: Cloud infrastructure dynamically scales resources based on workload demands, optimizing performance and cost-efficiency.
- Maintainability: Modern, modular architecture simplifies updates and reduces maintenance overhead.
- Enhanced Performance: PostgreSQL offers fast, reliable data access, while parallel processing accelerates EANN training and inference.
- Cost Optimization: Pay-as-you-go cloud services ensure efficient resource utilization, reducing operational costs.
- User-Centric Design: The React front-end provides a seamless, interactive user experience, enabling effective data exploration and analysis.

This phased approach ensures a robust transition from an on-premise system to Neuroid in the Cloud, leveraging cloud infrastructure to enhance scalability, reliability, and performance in processing parallel EANN workloads. As shown in Figure 8

Figure 8 User Interface Framework for Data visualization of Evolutionary Artificial Neuroidal



Input to EANN user interface is validated for schema parameters using validation logic build in the Data visualization framework as shown in Figure 8 above and Figure 9 shows the actual result for a sample input file validation.

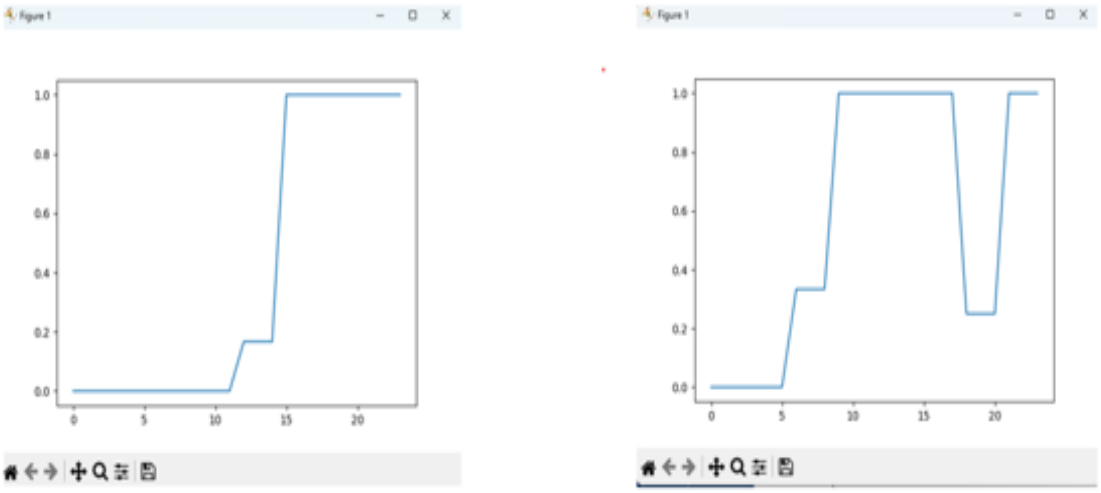
Figure 9 Data visualization for EANN for Input file and schema validation



Now the data processing, analyzing for neuroidal network and supervised training using Genetic Algorithm is running in the background thereby presenting results in the frontend. Figure 10 below shoes real-time output from the User Interface framework for EANN.

Figure 10 Real-time output for Evolutionary Artificial Neuroidal Network

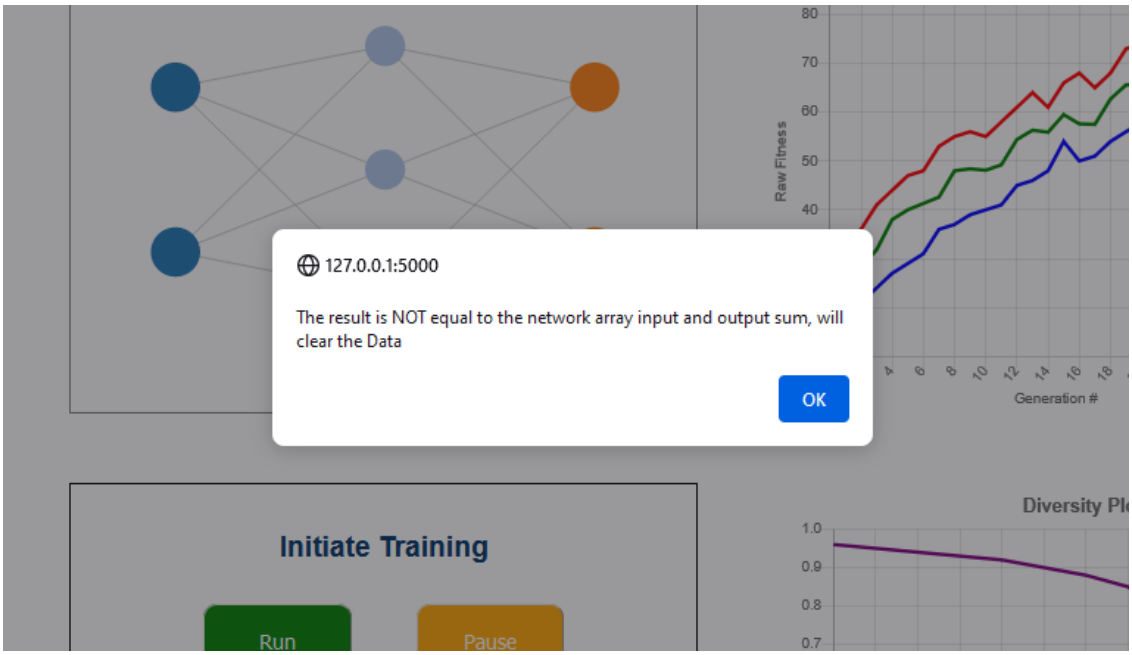
Real-time output with Output of run_neuroid function plotted for sample input test file with 2 inputs in Evolutionary Artificial Neuroidal Network.



The below figure 11 shoes the error handling logic built in for Data processing for EANN as results are processed in above step in figure 10 as shown.

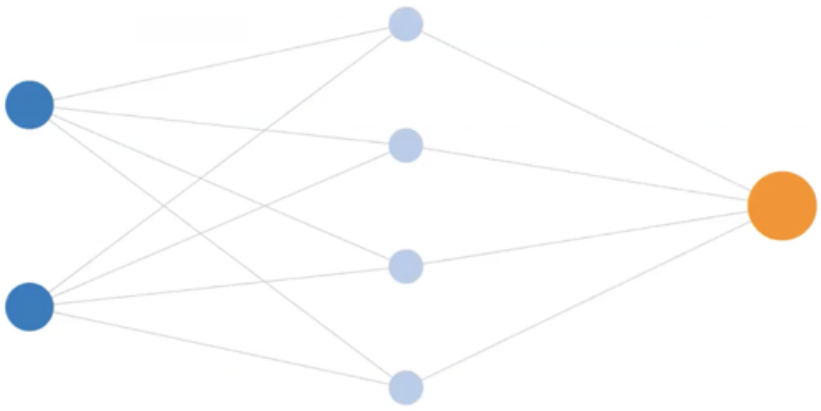
Figure 11 Error Detection pop-up functionality for EANN

To validate the submitted dataset is compatible with the set of Neuroid Network Framework.



The below figure 12 shows the Neuroid Network Genome representing a sample network of (2,4,1) with two neuroids at input layer, four neuroids in hidden layer (in this example we have one hidden layer, and it can be more than one hidden layer) and one neuroid at the output layer.

Figure 12 Neuroid Network Genome sample:



The Fitness and Diversity of the Evolutionary Artificial Neuroidal Network is plotted against the Fitness across Generation and Diversity across Generation where in the thresholds are set at user interface layer using the Data Visualization framework. These setting values for threshold are used across multiple generations until the cutoff is met and convergence occurs. In some cases the convergence does not occur even after multiple generation of processing and supervised training which is a rare case of input values used for Neuroid Network Genome. As displayed in below Figure 13 for Fitness and the EANN diversity across generation in Figure 14.

Figure 13 Fitness Plot across generation for evolutionary artificial neuroidal network

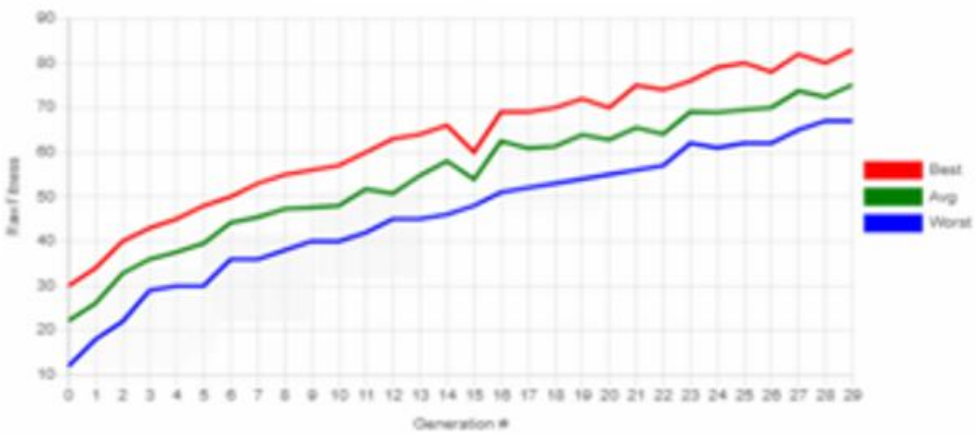
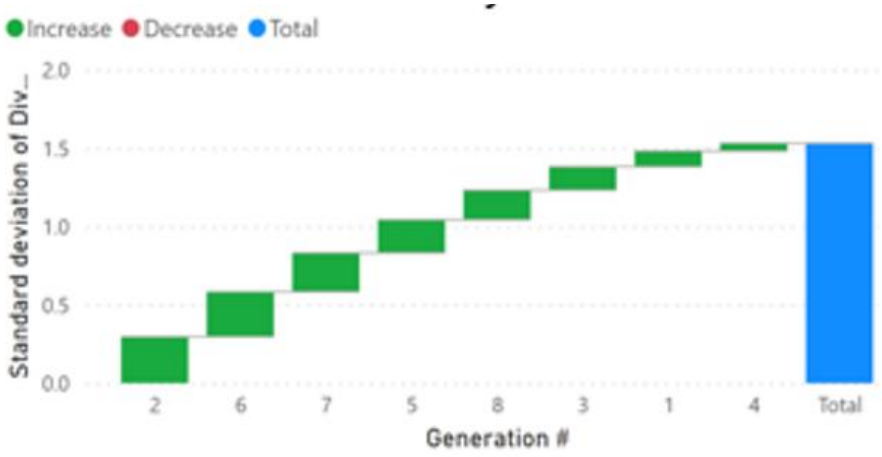


Figure 14 Diversity plot across generations for evolutionary artificial neuroidal network



Significance

Cloud Computing for Parallel Processing of Evolutionary Artificial Neuroidal Networks (EANN) Using Serverless Architecture: This research presents a transformative approach to deploying and managing Evolutionary Artificial Neuroidal Networks (EANN) within a cloud environment, leveraging serverless architecture to enhance scalability, cost-efficiency, and performance. The inherent flexibility and elasticity of cloud infrastructure provide a dynamic foundation for executing parallel processing tasks with minimal manual intervention. By utilizing serverless computing, resources are allocated dynamically, ensuring optimal utilization and significant cost savings. This paradigm not only reduces operational complexity but also accelerates processing times, empowering organizations to implement advanced machine-learning solutions with higher agility and lower overhead costs.

End-to-End Implementation of Parallel Processing of EANN with Serverless Architecture and MVC Three-Tier Structure: Implementing an MVC three-tier structure within the context of EANN processing establishes a robust, modular, and maintainable framework. This architectural approach effectively separates concerns, streamlining development, maintenance, and scalability. The transition from an on-premise solution to a cloud-native architecture highlights the adaptability of the proposed system. By incorporating modern technologies such as PostgreSQL, React, and Flask, the cloud-native implementation ensures enhanced performance, scalability, and user experience. This dual-phase development strategy provides a flexible and secure solution capable of managing large-scale data processing and analysis tasks efficiently.

Conclusion

This research presents a comprehensive solution for addressing the scalability, cost-efficiency, and operational complexity challenges inherent in deploying Evolutionary Artificial Neuroidal Networks (EANNs) by integrating cloud computing, serverless architecture, and an MVC three-tier structure. Traditional machine learning systems, particularly those based on Perceptron-based architectures, have struggled to meet the demands of modern, large-scale, and dynamic real-world applications. This study demonstrates that a cloud-native approach leveraging serverless computing and parallel processing significantly mitigates these limitations, offering a scalable, cost-effective, and operationally efficient framework.

The adoption of serverless architecture introduces dynamic resource allocation and elasticity, ensuring seamless scalability while eliminating the need for complex infrastructure management. The integration of parallel processing further accelerates training and inference processes, optimizing resource utilization and reducing computational overhead—key bottlenecks in handling high-dimensional and continuously evolving datasets. This combination establishes a robust foundation for addressing the high computational demands of EANNs while maintaining cost-effectiveness.

The implementation of an MVC three-tier structure enhances the modularity, maintainability, and adaptability of the proposed solution. By separating the presentation layer, business logic, and data layer, the MVC framework facilitates continuous development, system upgrades, and streamlined management, making it particularly well-suited for both on-premise and cloud-native environments. This architecture ensures long-

term scalability and maintainability, addressing challenges often associated with monolithic machine learning deployments.

Furthermore, the cloud-native migration utilizing technologies such as PostgreSQL, React, and Flask positions the solution as a future-ready framework capable of adapting to evolving technological landscapes. By leveraging cloud-native infrastructure, this research establishes a flexible and resilient system that supports real-time learning and processing while maintaining operational efficiency.

In summary, this research effectively addresses the key limitations of traditional machine learning systems by integrating serverless architecture, parallel processing, and an MVC three-tier framework within a cloud-native ecosystem. The proposed approach offers a transformative, scalable, and cost-effective infrastructure for deploying EANNs, capable of meeting the demands of complex, dynamic, and high-dimensional real-world applications. This study establishes a solid foundation for future advancements in machine learning, enabling more efficient, adaptable, and transparent systems for real-time data processing and analysis.

Recommendations for Future Work

To further enhance the proposed architecture's capabilities and practical applicability, several avenues for future research and development can be explored. One important direction is the refinement of resource allocation algorithms and the development of advanced auto-scaling policies. Incorporating real-time workload analysis and adaptive scaling mechanisms could significantly improve the performance and resource efficiency of serverless environments.

Integrating the architecture with emerging cloud-native services offers another critical opportunity for improvement. Advanced machine learning pipelines, container orchestration frameworks like Kubernetes, and automated CI/CD processes could be incorporated to streamline system deployment, monitoring, and management. These enhancements would result in more seamless and efficient operational workflows.

Security remains a cornerstone of cloud computing, and the architecture would benefit from the implementation of robust security measures such as end-to-end encryption, role-based access control (RBAC), and continuous compliance auditing. These strategies are essential for safeguarding data and ensuring regulatory adherence, particularly in complex and dynamic cloud ecosystems.

On the user experience front, enhancing the React-based front end by introducing real-time dashboards, interactive analytics, and advanced visualization tools could significantly improve the system's usability. Such improvements would facilitate a more intuitive interpretation of Evolutionary Artificial Neural Network (EANN) results, thereby elevating the overall user experience.

Another important area for development is cross-platform compatibility. Creating solutions that enable seamless interoperability across multiple cloud providers would address vendor lock-in challenges and improve system redundancy, reliability, and resilience. A multi-cloud strategy would also position the architecture to meet diverse operational requirements more effectively.

Conducting industry-specific case studies could provide valuable insights into the architecture’s practical applications. These studies would help identify domain-specific challenges and optimization opportunities, validating the system's adaptability and impact across various sectors.

To ensure the longevity and relevance of EANN models, implementing robust lifecycle management practices is crucial. Automated versioning, real-time monitoring, and periodic retraining will help maintain model accuracy, relevance, and performance throughout their operational lifecycle.

Addressing current limitations in serverless frameworks is another essential consideration. Research focusing on reducing cold start latencies and optimizing orchestration for large-scale parallel processing would enhance the system’s responsiveness and scalability.

Expanding the architecture’s compatibility to support a broader range of machine learning frameworks and libraries would significantly increase its versatility. This flexibility would allow organizations to employ the most suitable tools for their specific applications, fostering innovation and adaptability in various machine-learning projects.

Finally, developing comprehensive data governance policies and compliance frameworks tailored to serverless environments is critical. Such measures are particularly crucial for

industries like healthcare, finance, and government, where data security and regulatory adherence are paramount.

By addressing these areas, the proposed architecture has the potential to evolve into a highly robust, secure, and scalable solution. These advancements would empower organizations to fully harness the potential of serverless computing and cloud-based solutions, driving innovation and operational excellence in the application of Evolutionary Artificial Neural Networks.

References

Alzahrani, R. A., & Parker, A. C. (2020, July 27-29). *Neuromorphic circuits with neural modulation enhance the information content of neural signaling.* International Conference on Neuromorphic Systems (ICONS) 2020. https://doi.org/10.1145/3407197.3407204.

Rachel Goeken, Silva, R, (2022). Computational Simulations for Neuroids and Neuroidal Networks.

Argüello, E., Silva, R., Castillo, C., & Huerta, M. (2012). The neuroid: A novel and simplified neuron-model. *2012 Annual International Conference of the IEEE Engineering in Medicine and Biology Society*, 1234-1237. https://doi.org/10.1109/embc.2012.6346160.

Argüello, E., Silva, R., & Gruber, A. (2013, July 3-7). Development of a linear induction motor system-based artificial muscle. *35th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC) 2013*, 6211-6214. https://doi.org/10.1109/EMBC.2013.6610972.

Argüello, E., Silva, R., Huerta, M., & Avila, R. (2015). Computational modeling of peripheral pain: A commentary. *BioMedical Engineering Online, 14*(56). https://doi.org/10.1186/s12938-015-0049-x.

Argüello, E., Silva, R., Huerta, M., & Castillo, C. (2013, July 3-7). New trends in computational modeling: A retina neuroid-based model. *35th Annual International*

Conference of the IEEE Engineering in Medicine and Biology Society (EMBC) 2013, 4561-4564. https://doi.org/10.1109/EMBC.2013.6610562.

Argüello, E., Silva, R., Huerta, M., & D'Alessandro Martínez, A. (2015, June). Lamina specific loss of inhibition may lead to distinct neuropathic manifestations: A computational modeling approach. *Research on Biomedical Engineering, 31*(2). https://doi.org/10.1590/2446-4740.0734.

Gartner. (n.d.). Definition of neural network. *Gartner Information Technology*. https://www.gartner.com/en/information-technology/glossary/neural-net-or-neural-network.

Freund, Y., & Schapire, R. E. (1999). Large margin classification using the perceptron algorithm. *Machine Learning, 37*, 277-296. https://doi.org/10.1023/A:1007662407062.

Goeken, R., & Ulsh, J. (2022). Documentation of developing computational simulations for neuroids and neuroidal networks [Unpublished manuscript]. Villanova University.

Hopfield, J. J. (1982). Neural networks and physical systems with emergent collective computational abilities. *Proceedings of the National Academy of Sciences of the United States of America, 79*(8), 2554-2558. https://doi.org/10.1073/pnas.79.8.2554.

Mitchell, M. (1996). *An introduction to genetic algorithms* (1st ed.). Prentice Hall of India.

Mühlenbein, H. (1990). Limitations of multi-layer perceptron networks—Steps towards genetic neural networks. *Parallel Computing, 14*(3), 249-260. https://doi.org/10.1016/0167-8191(90)90079-O.

Silva, R. (2005). Study of a pacing neuroblastoma cell line via a novel macroelectrode technique [Doctoral thesis, Pennsylvania State University]. *Electronic Theses and Dissertations for Graduate School*. https://etda.libraries.psu.edu/catalog/6759.

Silverthorn, D. U. (1998). *Human physiology: An integrated approach* (1st ed.). Prentice Hall.

Simplilearn. (2009-2023). What is perceptron: A beginner's guide for perceptron. https://www.simplilearn.com/tutorials/deep-learning-tutorial/perceptron.

Appendix A

This Appendix shows the full Neuroid class code as it currently stands.

```
Class Neuroid:
    def __init__(self, umbr, beta, kr, maxcount, t, log=False):
        self.log = log

        # Neuroid parameters
        self.umbr = umbr
        self.beta = beta
        self.kr = kr
        self.maxcount = maxcount
        self.t = t

        # Internal state variables
        self.count1 = 0
        self.count2 = 0

        self.y = 0
        self.nt_out = 0
        self.time = 0

        # Data storage for logging
        self.sum_stream = []
        self.count1_stream = []
        self.count2_stream = []
        self.y_stream = []
        self.nt_out_stream = []

    def run_comparator(self, inputs, weights):
        input_sum = round(sum(inputs), 3)

        # Comparator logic based on input sum and thresholds
        if input_sum > self.umbr:
            if self.count1 > self.beta / (input_sum - self.umbr):
                self.count1 = 0
            else:
                self.count1 += 1
        else:
            self.count1 = 0

        # Logging the internal states
        if self.log:
            self.sum_stream.append(input_sum)
            self.count1_stream.append(self.count1)

    def run_freq_modulator(self):
        # Frequency modulation logic based on count1
        if self.count1 == 1:
```

```
            self.y = 1
        else:
            self.y = 0

        # Logging the output state of modulation
        if self.log:
            self.y_stream.append(self.y)

    def run_freq_demodulator(self):
        # Frequency demodulation logic based on the modulated signal
        if self.y == 1:
            self.nt_out = self.kr / self.count2
            self.count2 = 0
        else:
            self.count2 += 1

        # If count2 exceeds maxcount, reset output
        if self.count2 > self.maxcount:
            self.nt_out = 0

        # Logging the demodulated output
        self.nt_out_stream.append(self.nt_out)

        if self.log:
            self.count2_stream.append(self.count2)

    def get_axon_signal(self, inputs, weights):
        # Validate the input and weight sizes
        if len(inputs) != len(weights):
            raise Exception("Size of inputs and size of weights must be
the same!")

        y_t = []
        # Process inputs and weights for the given time window 't'
        for i in range(len(weights) - self.t + 1):
            self.run_comparator(inputs[i:(i + self.t)], weights[i:(i +
self.t)])
            self.run_freq_modulator()
            y_t.append(self.y)
        return y_t

    def run_neuroid(self, inputs, weights):
        # Validate the input and weight sizes
        if len(inputs) != len(weights):
            raise Exception("Size of inputs and size of weights must be
the same!")

        # Run the neuroid processing for the input-weight pairs
        for i in range(len(weights) - self.t + 1):
```

```
            self.run_comparator(inputs[i:(i + self.t)], weights[i:(i +
self.t)])
            self.run_freq_modulator()
            self.run_freq_demodulator()
            self.time += 1

            # Log the internal state at each time step if logging is
enabled
            if self.log:
                print("======= Time", self.time, "=======")
                print("nt_out =", self.nt_out)
                print("y =", self.y)
                print("count1 =", self.count1)
                print("count2 =", self.count2)
                print()

        return self.nt_out_stream
```

Appendix B

Code for Diversity Chart In User Interface:

```
// this is mock data, eventually call a different function to get the
diversity metrics
// var diversity = get_diversity();
var diversity = [.96,.95,.94,.93,.92,.90,.88,.85,.75,.65,.40];
// generate list from 0 to the last generation for the X-axis
var xValues = [...Array(diversity.length).keys()];

var myChart = new Chart("DiversityPlot", {
    type: "line",
    data: {
        labels: xValues,
        datasets: [{
            data: diversity,
            pointRadius: 0,
            borderColor: "purple",
            lineTension: 0,
            fill: false
        }]
    },
    options: {
        title: {
            display: true,
            text: "Diversity Plot",
            fontSize: 16
        },
        legend: { position: "right" },
        tooltips: { mode: "index", intersect: false },
        hover: { mode: "index", intersect: false },
        scales: {
            yAxes: [{
                scaleLabel: { display: true, labelString: "Diversity"
},
                ticks: {min: 0, max: 1} }],
            xAxes: [{ scaleLabel: { display: true, labelString:
"Generation #" } }]
        }
    }
});
```

Appendix C

Code for Fitness Chart:

```
var fitness_scores = []

// this is mock data creation future work would load the scores in as
an array here //
// fitness_scores is a 2d array of numbers where the 0th array holds
each individual's fitness score from the 0th generation.
// var fitness_scores = get_fitness_scores();
for(var i = 1; i < 31; i++) {
    fitness_scores.push(Array.from({length: 10}, () =>
Math.floor(Math.random()*20+12*Math.sqrt(i))));
}

// calculate the best, average, and worst for each generation and store
to be displayed
var best = [], avg = [], worst = [];
fitness_scores.forEach(function(d) {
    best.push(Math.max(...d));
    worst.push(Math.min(...d));
    avg.push(d.reduce((a, b) => a + b, 0) / d.length);
});

// generate list from 0 to the last generation for the X-axis
var xValues = [...Array(best.length).keys()];

// create fitness plot with three lines on it
var myChart = new Chart("FitnessPlot", {
    type: "line",
    data: {
        labels: xValues,
        datasets: [{
            data: best,
            pointRadius: 0, // remove each generation's dot
            label: "Best", // shows up in the hover and legend
            borderColor: "red", // color of the line
            lineTension: 0, // 0 means straight between each dot,
increase to smooth line into bends
            fill: false // fills beneath the line
        }, {
            data: avg,
            pointRadius: 0,
            label: "Avg",
            borderColor: "green",
            lineTension: 0,
            fill: false
         }, {
            data: worst,
```

```
            pointRadius: 0,
            label: "Worst",
            borderColor: "blue",
            lineTension: 0,
            fill: false
        }]
    },
    options: {
        title: {
            display: true,
            text: "Fitness Plot",
            fontSize: 16
        },
        tooltips: { mode: "index", intersect: false }, // displays box
with values, doesn't need to be intersecting a point
        hover: { mode: "index", intersect: false }, // displays point
on line, doesn't need to be intersecting the point
        legend: { position: "right" },
        scales: {
            yAxes: [{ scaleLabel: { display: true, labelString: "Raw
Fitness" } }],
            xAxes: [{ scaleLabel: { display: true, labelString:
"Generation #" } }]
        }
    }
});
```

Appendix D

Code for generating Evolutionary Artificial Neural Network Chart:

```
var
height=document.getElementById("neural_network_wrapper").offsetHeight;
var
width=document.getElementById("neural_network_wrapper").offsetWidth;
var nodeSize=30;
var networkInputBox = document.getElementById("vector");
var modelInitFile = document.getElementById("")

// listen for input to the network structure input box, draw the
network accordingly
networkInputBox.addEventListener("input", (e) => {
    if(networkInputBox !== null) {
        var parsed_input = networkInputBox.value.split(",");
        drawGraph(parsed_input);
    }
});

// calculate how big the nodes should be in the display pane depending
on how many there are
function layerSize(layer, network) {
    return parseInt(30/Math.sqrt(network[layer]));
}

// draw the graph based on the input, using edges and weights if
provided, otherwise fully connected and all weights = 1
function drawGraph(networkInput, edges, weights) {
    try {
        d3.select("#neural_network").remove(); // remove the old
network if it was previously drawn
    } catch {}
    var color = d3.scale.category20();
    var svg = d3.select("#neural_network_wrapper")
    .append("svg")
    .attr("id","neural_network");

    var nodes = [];
    for(var layer_i=0; layer_i < networkInput.length; layer_i++) {
        for(var node_i=0; node_i < networkInput[layer_i]; node_i++) {
            nodes.push({"layer": layer_i});
        }
    }

    // initialize netsize layers and node positions within the layer
    var netsize = {};
    nodes.forEach(function (d) {
```

```
      if(d.layer in netsize) {
          netsize[d.layer] += 1;
      } else {
          netsize[d.layer] = 1;
      }
      d["lidx"] = netsize[d.layer];
    });

    // calc distances between nodes
    var largestLayerSize = Math.max.apply(
        null, Object.keys(netsize).map(function (i) { return
netsize[i]; }));

    var xdist = width / Object.keys(netsize).length;

    // create node locations
    nodes.map(function(d) {
      d["x"] = (d.layer+0.5) * xdist;
      d["y"] = (d.lidx / (netsize[d.layer]+1)) * height;
    });

    // generate edges in graph
    var links = [];
    if(edges === undefined) {
        // autogenerate fully connected network links if no edges
provided
        nodes.map(function(d, i) {
            if (d.layer < networkInput.length-1 &&
networkInput[d.layer+1] !== "") {
                var start_next = d3.sum(networkInput.slice(0,
d.layer+1))
                var end_next = d3.sum(networkInput.slice(0, d.layer+2))
                var outgoing_edges = Array(end_next -
start_next).fill().map((_, idx) => start_next + idx)
                outgoing_edges.forEach(function(out_e) {
                    links.push({"source": i, "target": out_e, "value":
1})
                });
            }
        });
    } else {
        // check to see if we were given weights too, if not just
assume 1
        if(weights === undefined) {
            var weights = new Array(edges.length).fill(1);
        }
        edges.map(function(e, i) {
            var parsed_edges = e.split(" ");
            var starting_node = d3.sum(networkInput.slice(0,
parsed_edges[0]))+parseInt(parsed_edges[1]);
```

```
            var ending_node = d3.sum(networkInput.slice(0,
parsed_edges[2]))+parseInt(parsed_edges[3]);
            links.push({"source": starting_node, "target": ending_node,
"value": Math.min(20*weights[i])})
        })
    }

    // draw links between all nodes (fully connected network)
    var link = svg.selectAll(".link")
        .data(links)
      .enter().append("line")
        .attr("class", "link")
        .attr("x1", function(d) { return nodes[d.source].x; })
        .attr("y1", function(d) { return nodes[d.source].y; })
        .attr("x2", function(d) { return nodes[d.target].x; })
        .attr("y2", function(d) { return nodes[d.target].y; })
        .style("stroke-width", function(d) { return Math.sqrt(d.value);
});

    // draw nodes
    var node = svg.selectAll(".node")
        .data(nodes)
      .enter().append("g")
        .attr("transform", function(d) {
            return "translate(" + d.x + "," + d.y + ")"; }
        );
    var circle = node.append("circle")
        .attr("class", "node")
        .attr("r", function(d) { return layerSize(d.layer,
networkInput); })
        .style("fill", function(d) { return color(d.layer); });
}
```

Appendix E

Code for Neuroid API Routes:

```
from flask import Blueprint, render_template, request, redirect,
url_for, current_app
import csv
from service.csv_parser import CsvParserService
from service.program_inputs_capture_service import
ProgramInputCaptureService
from service.inputs_processing_service import InputProcessingService


# Create a blueprint for the navigation routes
neuroid_api_bp = Blueprint('neuroid_api', __name__)

# Define the route for the /reset API
@neuroid_api_bp.route("/reset")
def reset():

    return redirect("/index")



"""Route used to capture user's data"""
@neuroid_api_bp.route("/systemInput" , methods=['POST'])
def system_input():
    print("THIS IS ME TESTING")
    print(request.form)

    # Can also be invoked statically
    programInputCaptureService = ProgramInputCaptureService()
    programInputCaptureService.process_user_input(request.form,
request.files, current_app.config["user_input_instance"] );

    return redirect("/results")


"""Display the information entered by the user"""
@neuroid_api_bp.route("/results")
def results():

    user_input= current_app.config["user_input_instance"]
    print("THIS IS ME TESTING AGAIN")
    print("Final user input data:")
    print(user_input)

    return render_template('results.html',
vector=user_input.get_vector(), feedback=user_input.get_feedback(),
```

```
mutations=user_input.get_mutations(),
transposes=user_input.get_transposes(),
                           survival=user_input.get_survival(),
population=user_input.get_population(),
fitness=user_input.get_fitness() , trainingDataFile =
user_input.get_trainingDataFile(), trainingData=
user_input.get_trainingData())

"""Display the information entered by the user"""
@neuroid_api_bp.route("/calculations", methods=['GET'])
def calculations():
    print("inside calculations...")
    inputProcessingService = InputProcessingService();

    inputProcessingService.update_data_for_neuroid(current_app.config["
user_input_instance"]);

    return redirect("/index");
```

Appendix F

Code for Navigation Routes:

```
from flask import Blueprint, render_template, request, redirect,
url_for, current_app

# Create a blueprint for the navigation routes
navigation_bp = Blueprint('nav', __name__)

# Define the route for the root URL
@navigation_bp.route("/", methods=['GET'])
def root():
    return redirect("/index")

# Define the route for the /index URL
@navigation_bp.route("/index", methods=['GET'])
def index():
    print("after index load")
    print(current_app.config["user_input_instance"])
    user_input= current_app.config["user_input_instance"]

    return render_template("index.html" ,
vector=user_input.get_vector(), feedback=user_input.get_feedback(),
mutations=user_input.get_mutations(),
transposes=user_input.get_transposes(),
                           survival=user_input.get_survival(),
population=user_input.get_population(),
fitness=user_input.get_fitness() , trainingDataFile =
user_input.get_trainingDataFile(), trainingData=
user_input.get_trainingData())
```

Code to Generation Population:

```
class Population:
    """
    This class contains many Networks, along with the information
needed for things such as instantiating
    the networks, evaluating their performance, survival rate,
mutation, etc.
    """
    # Responsible with keeping track across generations. Including
initializing the first generation, then the genetic algorithm used to
create the new networks
    def __init__(self,
                 population_size,
                 fitness_function,
                 network_shape,
                 network_edges,
```

```
                filename,
                desired_accuracy=0.80,
                survival_rate=0.5,
                mutation_rate=0.01,
                training_split=0.8,
                umbr=1,
                beta=10,
                kr=5,
                maxcount=90,
                t=1,
                log=False):
        """
        Creation of the Population.

        Required Parameters:
        - population_size (int)
          - Number of networks to have in each generation, a higher
value results in greater diversity but higher training time
        - fitness_function (func)
          - Loss function to use for evaluating the network
performances, lower values are better
        - network_shape (list)
          - Values determining the number of nodes in each layer:
[2,4,5,2] represents a 4 layer network with 2 input nodes,
            the first hidden layer containing 4 nodes, the second
hidden layer containing 5 nodes, and an output layer of 2 nodes
        - network_edges (list(list))
          - 2D list denoting the nodes which the i'th node sends an
edge to

        Optional Paramters:
        - desired_accuracy (float)
          - Accuracy at which to end training at
        - survival_rate (float)
          - Percentage of networks which should survive each generation
        - mutation_rate (float)
          - Percentage of neuroids which should mutate at each breeding
between two networks
        - training_split (float)
          - Percentage of data which should be used for training, 1-
training_split is reserved for testing
        - umbr (int)
          - Minimum threshold of activation (Pulse Frequency
Modulation)
        - beta (int)
          - Proportionality constant (Pulse Frequency Modulation)
        - kr (int)
          - Regeneration constant (Pulse Defrequency Modulation)
        - maxcount (int)
```

```
          - Maximum value for regeneration constant (Pulse Defrequency
Modulation)
        - t (int)
          - Time constant to use for overlap - currently always at 1
        """

        # Required parameters
        self.population_size = population_size
        self.fitness_function = fitness_function
        self.network_shape = network_shape
        self.filename = filename

        # Optional parameters
        self.desired_accuracy = desired_accuracy
        self.survival_rate = survival_rate
        self.mutation_rate = mutation_rate
        self.training_split = training_split
        self.umbr = umbr
        self.beta = beta
        self.kr = kr
        self.maxcount = maxcount
        self.t = t
        self.log = log
        if network_edges:
            self.network_edges = network_edges
        else:
            self.network_edges =
generate_fully_connected_feedforward_edges(network_shape)

        # Calculated attributes
        self.curr_networks = []
        self.training_data = None
        self.out_training_data = None
        self.testing_data = None
        self.out_testing_data = None
        self.curr_generation = 0
        self.fitness_scores = []
        self.diversity_scores = []
        self.best_accuracy = []

        # Initialize the networks in the population
        for i in range(self.population_size):
            self.curr_networks.append(Network(self.network_shape,
self.network_edges, self.umbr, self.beta, self.kr, self.maxcount,
self.t))

        # read in the data provided with the training_split requested
        self.read_data(self.filename, self.training_split)

    def read_data(self, filename, training_split):
```

```
        """Reads the CSV of training data and randomly splits
accordingly into training and testing samples. Removes first row."""
        print("beginning reading data...")
        with open(filename, 'r') as training_file:

            # instantiate the file reader for the CSV
            file_reader = csv.reader(training_file)

            # remove the first row since it's the column headings
            next(file_reader)

            # cast the string bits into integers
            file = [[[int(c) for c in var_i] for var_i in
training_sample] for training_sample in file_reader]

            # use sklearn's train_test_split to partition the
training/testing in a fast manner
            train, test = train_test_split(np.asarray(file),
train_size=int(round(training_split*len(file))), random_state=42)

            self.training_data = train
            self.testing_data = test

            training_file.close() # close the file

        print(f"read {len(file)} samples, split into {len(train)}
training and {len(test)} testing.")
        return self.training_data

    def get_network_max_fitness_score(self):
        """Returns the index of the network which has the highest
fitness score, ties broken by lower index."""
        # tracking the score and index of score
        max_fitness_score, max_i =
self.fitness_scores[self.curr_generation][0], 0

        # for each index and net at that index, if it has a higher
fitness score, replace current, then return final index
        for i, net in enumerate(self.curr_networks):
            fit_score_i =
net.get_fitness_accuracy_scores(self.training_data,
self.fitness_function)[0]
            if fit_score_i > max_fitness_score:
                max_fitness_score = fit_score_i
                max_i = i
        return max_i

    def kill_old(self):
        """Uses self.survival_rate and Network.get_fitness_score to
remove the worst networks from the current population"""
```

```
        if self.log:
            print("killing old population...")
        num_surviving = int(self.survival_rate*self.population_size)

        # for the number of networks which need to be killed, delete
the worst performing network according to fitness score
        for i in range(len(self.curr_networks)-num_surviving):
            del
self.curr_networks[self.get_network_max_fitness_score()]

    def create_new_population(self):
        """Fill in the population until carrying capacity is
reached."""
        if self.log:
            print("creating new population...")
        # number of living parents (survival_rate*population_size)
        num_parents = len(self.curr_networks)

        # while we are not at the capacity, choose two random parents
and create a baby network with them
        while len(self.curr_networks) != self.population_size:
            parent_1, parent_2 = random.sample(range(num_parents), 2)
            self.curr_networks.append(create_baby(self.curr_networks[pa
rent_1], self.curr_networks[parent_2], self.mutation_rate))

    def calculate_diversity(self):
        """TO DO: Find a way to measure the diversity among the
population and calculate then return here."""
        return 5

    def generation_step(self):
        """One generation of the population lives and dies here."""
        if self.log:
            print("calculating fitness scores")

        # Calculate the fitness and accuracy scores for each network in
the population
        fitness_accuracy_scores =
[[net.get_fitness_accuracy_scores(self.training_data,
self.fitness_function)] for net in self.curr_networks]

        # Save all the fitness scores and the best accuracy, then
calculate and store the diversity score
        self.fitness_scores.append([score[0][0] for score in
fitness_accuracy_scores])
        self.best_accuracy.append(max([score[0][1] for score in
fitness_accuracy_scores]))
        self.diversity_scores.append(self.calculate_diversity())

        print(self.send_logs())
```

```
        # Kill off the worst populations and re-fill with new baby
networks
        self.kill_old()
        self.create_new_population()

    def send_logs(self):
        """TO DO: Link to front end for sending logs in progress of
training."""
        return {"generation": self.curr_generation,
                "fitness":
sum(self.fitness_scores[self.curr_generation]) /
len(self.fitness_scores[self.curr_generation]), # Calculates the
average fitness score
                "diversity":
self.diversity_scores[self.curr_generation],
                "best": self.best_accuracy[self.curr_generation]}

    def train(self, desired_accuracy):
        """Main method called to train until desired_accuracy is
reached."""
        print("training beginning...")

        # Initial generation, then continue until we pass the desired
accuracy
        self.generation_step()
        while self.best_accuracy[self.curr_generation] <
self.desired_accuracy:
            self.curr_generation += 1
            self.generation_step()

        # find all networks with accuracy over the threshold, then
return the one which has the highest fitness score as tiebreaker
        acceptable_networks = [net for net in self.curr_networks if
net.get_accuracy(training_data) == self.best_accuracy]
        best_network = [net for net in acceptable_networks if
net.get_fitness_score(training_data) ==
min([net.get_fitness_score(training_data) for net in
acceptable_networks])]
        return best_network[0]
```

Glossary

Neuroid – one processing unit

Neuroid Framework – Three-Tier Architecture Framework

Neuroids – multiple processing units, not necessarily connected

Neuroidal framework – multiple processing units connected to process data

AI - Artificial Intelligence

ANNG - Artificial Neuroid Network Genome

EANN - Evolutionary Artificial Neuroid Network

UI/UX: User Interface / User Experience

GA: Genetic Algorithm

ProQuest Number: 32238555

INFORMATION TO ALL USERS
The quality and completeness of this reproduction is dependent on the quality
and completeness of the copy made available to ProQuest.



Distributed by
ProQuest LLC a part of Clarivate (2025).
Copyright of the Dissertation is held by the Author unless otherwise noted.

This work is protected against unauthorized copying under Title 17,
United States Code and other applicable copyright laws.

This work may be used in accordance with the terms of the Creative Commons license
or other rights statement, as indicated in the copyright statement or in the metadata
associated with this work. Unless otherwise specified in the copyright statement
or the metadata, all rights are reserved by the copyright holder.

ProQuest LLC
789 East Eisenhower Parkway
Ann Arbor, MI 48108 USA